

Q.27 List and explain any two file allocation mechanisms.

[SPPU : May-19, End Sem, Marks 8]

Ans. : Refer Q.26.

Q.28 Explain advantages and disadvantages of contiguous and linked allocation methods.

Ans. : Advantages of contiguous allocation methods

1. It supports variable size portion.
2. Easy to retrieve single block.
3. Accessing a file is easy.
4. It provides good performance.

Disadvantages of contiguous allocation methods

1. It suffers from external fragmentation.
2. Pre-allocation is required.

Advantages of linked allocation methods

1. There is no external fragmentation.
 2. It is never necessary to compact disk space.
 3. Pre-allocation is not required.
- Disadvantages of linked allocation methods
1. Files are accessed only sequentially.
 2. Space required for pointers.
 3. Reliability is not good.
 4. Can not support direct access.

END... ✍

6

Systems Software and its Importance

Unit VI

6.1 : Study of Various Components of System Software

Q.1 Define system software. List example of system software.

Ans. : Definition : Systems software consists of a set of programs that support the operation of a computer system and help, the programmer, to simplify the programming process and create a environment to run application software efficiently.

• Examples of systems software are : *Text editors, Compilers, Loaders, Linker, Debugger, Assembler and Operating system.*

• Software is of two types : *System software and application software.*

Q.2 What is debugging? Explain classification of debugger.

Ans. : • Debugging means locating and then removing bugs, i.e., faults, in programs. A debugger is a tool that allows user to examine the state of a running program.

• An interactive debugging system provides programmers with facilities that aid in testing and debugging of programs. Debugging is a methodical process of finding and reducing the number of bugs, or defects, in a computer program.

• Classification of Debuggers are as follows :

1. **Static debugger** : Static debugger is not depends upon any specific software. User performed debugging task itself.
2. **Dynamic debugging** : Dynamic debugging is either software or hardware debugger. List of dynamic debugger are as follows :
 - a. **Breakpoint debugger** : Setting conditional and unconditional breakpoints at various positions in the program.

- b. **Kernel debugger** : Debugger with kernel debugging facilities is provided to debug the operating system.
- c. **Meta debugger** : Debugger provides debugging facility with meta debugging facility.
- d. **Debugger with same process** : Process of debugger and debuggee are same and both share the main memory also. This type of debugger is simple and easy to implement. This debugger is faster in execution.

Q.3 Explain system software in terms of program development environment.

Ans. : Systems Software : Program Development Environment

- **Text editor** : Software that permits the creation and editing of text files (i.e. application programs).
- **Compiler** : Translates programs written in a high level language to object code or machine code.
- **Assembler** : Translates programs written in assembly language to object code or machine code.
- **Static Linker** : Combines and resolves references between object programs and creates the executable code.
- **Debugger (Low-level)** : It is used to debug executable programs and their related object code and source program.

6.2 : Assemblers

Q.4 What is assembler? List assembler function.

- Ans. :** • The assembler is responsible for translating the assembly language program into machine code. When the source language is essentially a symbolic representation for a numerical machine language, the translator is called an **assembler** and the source language is called an assembly language.
- A pure assembly language is a language in which each statement produces exactly one machine instruction. Fig. Q.4.1 shows the role of assembler.

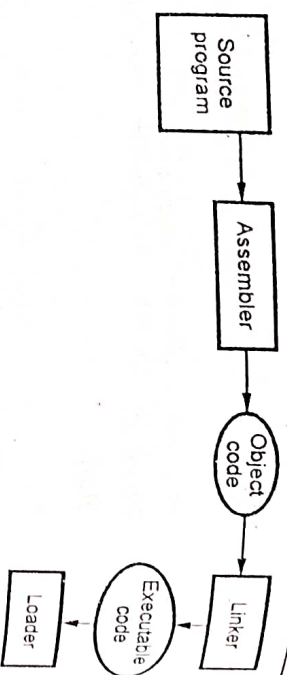


Fig. Q.4.1 Role of assembler

Assembler functions

1. Translate mnemonic opcodes to machine language.
2. Convert symbolic operands to their machine addresses.
3. Build machine instructions in the proper format.
4. Convert data constants into machine representation.
5. Error checking is provided.
6. Changes can be quickly and easily incorporated with a reassembly.
7. Variables are represented by symbolic names, not as the locations.

Q.5 With syntax and example types of assembly language statements

Ans. : It consists of three types of statements :

1. Imperative
2. Declaration
3. Assembler directives

imperative statement : It specifies an action to be performed during execution of the assembled program. Imperative programs focus on solving a problem using popular imperative languages such as C and Pascal based on side effects on memory. Each imperative statement translates into one machine instruction.

Declaration statement : It focuses on what the problem is and leaves the solution mechanism up to the language implementation. Within the declarative paradigm, most notable are the functional languages such as Lisp. Declarative languages are typically quite abstract and hence can be harder to implement efficiently.

Assembler Directive

Assembler directives instruct the assembler to perform certain actions during the assembly of a program. They can be used to declare variables, create storage space for results, to declare constants. The following assembler directives are used in the program :

1. **START** - Specify name and starting address for the program.
START < constant >
2. **END** - Indicate the end of the source program and specify the first executable instruction in the program.
END [< operand spec >]
3. **BYTE** - Generate character or hexadecimal constant, occupying as many bytes as needed to represent the constant.
4. **WORD** - Generate one word integer constant.

6.3 : Simple Assembly Scheme

Q.6 Explain steps for design specification of an assembler.

Ans. : • Following steps are used for the design specification of an assembler

1. Find out the information necessary to perform a task.
2. Specify data structure and define format of data structure.
3. To obtain and maintain the information, determine required processing.
4. To perform the task, determine required processing.

Q.7 Explain task of analysis phase and synthesis phase.

Ans. : Analysis phase task

1. Separate the label, mnemonic opcode and operand fields of a statement.
2. If statement contains label then the pair of (symbol, < LC content >) is entered in the symbol table.
3. Validity of the mnemonic opcode is checked through a look-up table by using mnemonics table.
4. Location counter processing is done.

Synthesis phase task

1. Read the machine opcode corresponding to the mnemonic from the mnemonics table.

2. Get an address of a memory operand from the symbol table.
3. Synthesize a machine instruction.

6.4 : Pass Structure of Assembler

Q.8 Describe two pass translation.

Ans. : • Forward reference is handled easily in two pass translation of the assembly language program. Location counter processing is performed in the first pass and symbols defined in the program are entered into the symbol table.

• Fig. Q.8.1 shows two pass structure.

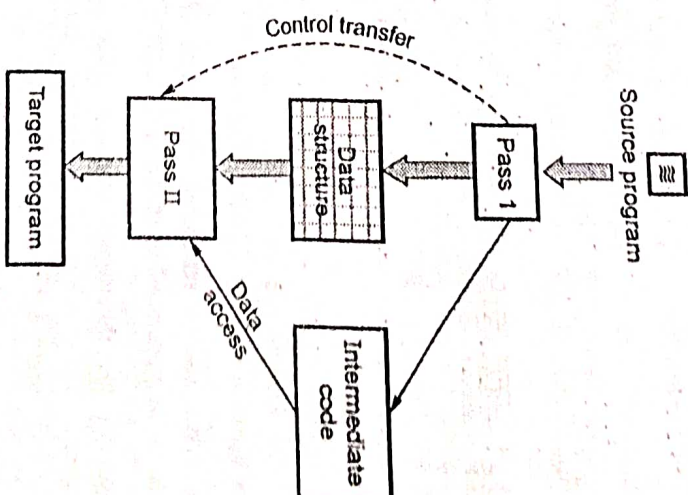


Fig. Q.8.1 Two pass assembly

- The second pass synthesizes the target from using the address information found in the symbol table.

- Functions of the two passes of a simple assembler are as follows.

1. Assign addresses to all statements in the program.
2. Save the values assigned to all labels for use in Pass 2.
3. Perform some processing of assembler directives.

Pass 2 (Assemble instructions and generate object program)

1. Assembler instructions (i.e. translating operation codes and looking up address)

2. Generates data values defined by BYTE, WORD etc.

3. Perform processing of assembler directives not done during Pass 1.

4. Write the object program and the assembly listing.

Q.9 Explain in detail Operation code table and symbol table.

Ans. : • Simple assembler uses two major internal data structures.

a) The Operation Code Table (OPTAB)

b) The Symbol Table (SYMTAB)

- OPTAB contains the mnemonic operation and its machine language equivalents. It also contains instruction format and length.

- OPTAB is used to look up mnemonic operation codes and translate them to their machine language equivalents. SYMTAB is used to store values assigned to labels.

- Location counter (LOCCTR) is also required. LOCCTR is a variable used to help in the assignment of addresses. It is initialized to the beginning address specified in the START statement.

- After processing every source statement, the length of the assembled instruction or data to be generated is added to LOCCTR

- OPTAB must contain the mnemonic operation code and its machine language equivalent. This table also contains information about instruction format and length for complex assembler.

- In Pass 1, OPTAB is used to look up and validate operation codes in the source program. In Pass 2 it is used to translate the operation codes to machine language.

DECODE

A Guide for Engineering Students

- OPTAB is usually organized as a hash table, with mnemonic operation code as the key. OPTAB is a static table, i.e. entries are not normally added to or deleted from it.

- SYMTAB includes the name and value for each label in the source program, together with flags to indicate error conditions. It also contains information about the data area or instruction labeled.

- During Pass 1, labels are entered into SYMTAB as they are encountered in the source program along with their assigned address.

- During Pass 2, symbols used as operands are looked up in SYMTAB to obtain the addresses to be inserted in the assembled instructions.

- SYMTAB is also organized as a hash table for efficiency of insertion and retrieval. Entries are rarely deleted from this table.

- It is possible for both passes of the assembler to read the original source program as input. Information such as location counter values and error flag statement can be communicated between the two passes. For this reason, Pass 1 usually writes an intermediate file that contains each source statement together with its assigned address, error indications etc. This file is used as the input to Pass 2.

6.5 : Design of Two Pass Assembler

Q.10 Briefly discuss advanced Assembler Directive.

Ans. : 1. ORIGIN (ORG)

- This can be used to indirectly assign values to symbols.

- When this statement is encountered during assembly of a program, the assembler resets its location counter to the specified value.

- The ORG statement will thus affect the values of all labels defined until the next ORG.

- Normally when an ORG without specified value is encountered, the previously saved location counter value is restored.

- Syntax of the ORIGIN directive is :

ORIGIN

<address spec>

DECODE

A Guide for Engineering Students

where <address spec> is an <operand spec> or <constant>.

- This type of statement is useful when the target program does not consist of consecutive memory words.
- No Forward Reference is allowed.

For example

SYMBOL : 6 bytes

a. VALUE : 1 word

c. FLAGS : 2 bytes

d. LDA VALUE, X

2. EQU

- Most assemblers provide an assembler directive that allows the programmer to define symbols and specify their values, for example.

The general form :

symbol EQU value

- One common use of EQU is to establish symbolic names that can be used for improved readability in place of numeric values.

- Another common use of EQU is in defining *mnemonic names* for registers.

For example :

A EQU 0

X EQU 1

L EQU 2

- These statements cause the symbols A, X, L, „ to be entered into SYMBOL with their corresponding values 0, 1, 2, ...

3. LTORG

- LTORG allows placing literals into a pool at some other location in the object program.
- Directive LTORG creates a literal pool that contains all of the literal operands used since the previous LTORG or the beginning of the program.

- Literals placed in a pool by LTORG will not be repeated in the pool at the end of the program.

- The LTORG statement permits a programmer to specify where literals should be placed. By default, assembler places the literals after the END statement.

- Assembler allocates memory to the literals of a literal pool. The pool contains all literals used in the program since the start of the program or since the last LTORG statement.

Q.11 What is forward reference ? How forward reference problem is solved ?

Ans. : • Once declared, identifiers can be used freely. Using an identifier before its declaration is called a forward reference. The use of a symbol at one point in the text which is not defined until some later point gives the wrong result.

- Forward references in programs cannot be resolved in a single pass. So we need more than one pass through the source code.

- General solution to the forward reference problem rests on the observation that all of the referenced symbols should be defined after processing a program even though they may not be defined when they are needed during processing.

- Therefore, if the output of the assembler after processing the input once is discarded, except for the contents of the symbol table, and then the input is processed again, with the symbol table already loaded.

- Forward reference is handled easily in two pass translation of the assembly language program.

- Location counter (LC) processing is performed in the first pass and symbols defined in the program are entered into the symbol table.

- In two pass translation, LC processing and construction of the symbol table is done. Back patching is used to solve the problem of forward reference.

- The operand field of an instruction containing a forward reference is left blank initially. The address of the forward reference symbol is put into this field when its definition is encountered.
- For example : **MOVER BREG, ONE**
- In the above statement, ONE is a forward reference. The memory location 101 contains the instruction opcode and address of BREG.
- Table of Incomplete Instructions (TII) is used for inserting the second operand address. TII contains the (< instruction address >, < symbol >)
- When assembler reads END statement, the symbol table would contain the addresses of all symbols defined in the source program and table of incomplete instruction would contain information describing all forward references.
- The assembler can now process each entry in TII to complete the concerned instruction.

Q.12 Explain difference between One Pass and Two Pass Assembler.

Ans. :

Sr. No.	One Pass	Two Pass
1.	It performs translation in one pass.	It performs translation in two pass.
2.	It scans entire file only once.	It requires two passes to scan source file.
3.	It generates intermediate code.	It does not generate intermediate code.
4.	Data Structure : Symbol table, literal table, Pool table and table of incomplete instructions.	Data Structure : Symbol table, literal table and Pool table
5.	Perform some processing of assembler directives	Perform processing of assembler directives not done in Pass-1

6.	It is faster than two pass assembler	It is slower than one pass assembler
7.	No object program is written; hence no loader is required.	Loader is required as object code is generated.

6.6 : Introduction to Macro Processors

Q.13 Define macro.

Ans. : Macro is a unit of specification for program generation through expansion. A macro consists of a name, a set of formal parameters and body of code.

Q.14 What are the difference between macro and functions ?

Ans. :

Macro	Function
Macro is Preprocessed	Function is Compiled
Macro does not support type checking	Function support type checking
Program length increases	Program length remain same
There is side effect while using macro.	No side effect while using function
Execution is Faster	Execution is slower
Useful where small source code appears many time	Useful where large source code appears many time
Macro does not Check Compile Errors	Function Checks Compile Errors
Before Compilation macro name is replaced by macro value	During function call, transfer of Control takes place

6.7 : Macro Expansion

Q.15 Explain the lexical expansion of Macro with example.

[SPPU : Aug.-14, Oct-16, In Sem, Marks 6]

Ans. : • Lexical expansion is replacement of a character string by another character string during program generation.

Lexical substitution consists of three types of strings

- a) Ordinary string
 - b) Name of a formal parameter which is preceded by the character '&'
 - c) Name of a preprocessor variable
 - d) In lexical expansion, only string type 1 is retained without substitution.
- Other two string types are replaced by the values of the formal parameters or preprocessor variables. Actual parameter string is used as the value of a formal parameter.
- Lexical expansion is to replace occurrences of formal parameters by corresponding actual parameters.

Example

- The following sequence of instructions is used to increment the value in a memory word by a constant.
1. Move the value from the memory word into a machine-register.
 2. Increment the value in the machine register.
 3. Move the new value into the memory word.
- Since the instruction sequence MOVE-ADD-MOVE may be used a number of times in a program, it is convenient to define a macro named INCR.
 - Using lexical expansion the macro call INCR A,B,AREG can lead to the generation of a MOVE-ADD-MOVE instruction sequence to increment A by the value of B using AREG to perform the arithmetic.
 - Use of Semantic expansion can enable the instruction sequence to be adapted to the types of A and B.
 - For example an INC instruction could be generated if A is a byte operand and B has the value 1.
 - If the parameters and arguments were associated with each other according to their positions in the macro prototype and the macro invocation statement, then these parameters in macro definitions are called as positional parameters.

- Syntax and written as

& < parameter name >

Example : &ROLLNO

where ROLLNO is the name of a parameter. In a positional parameter < parameter kind > is omitted. The < actual parameter spec > is a call on a macro using positional parameters is simply an < ordinary string >.

- The value of a positional formal parameter ABC is determined by the rule of positional association as follows :
1. In macro prototype statement, find original position of ABC.
 2. Also find the actual parameter specification occupying the same ordinal position in the list of the actual parameters in the macro call statement.

Q.16 Explain concept of keyword parameters.

Ans. : Keyword Parameters :

For keyword parameters,

< parameter name > $\Rightarrow$ ordinary string
 < parameter kind > $\Rightarrow$ string '='
 < actual parameter spec > $\Rightarrow$ written as
 < formal parameter name > = < ordinary string >

- Value of formal parameter is determined as follows :

1. Find the actual parameter specification which look likes XYZ = < ordinary string >
2. < ordinary string > is in the specification of the string ABC. So formal parameter value is XYZ = ABC

Default specifications of parameters

- Standard assumption value is a default value. This is suitable in such condition that, the parameter has the same value in most calls. When the desired value is different from the default value, the desired value can be specified explicitly in a macro call. Syntax for formal parameter specification is as follows :
- & < parameter name > | < parameter kind > | < default value > |]

Macros with mixed parameter lists

- Keyword parameters and positional parameters, both uses a macro. In this situation, all positional parameters must precede all keyword parameters. For example :

SUMUP X, Y, A = 10, B = n

- Here X and Y are positional parameters and A and B are keyword parameters and A and B are keyword parameters

6.3 : Introduction to Compilers**Q.17 What is compiler ?**

Ans. : **Definition** : Compiler is a program which takes one language (source program) as input and translates it into an equivalent another language (target program).

- During this process of translation if some errors are encountered then compiler displays them as error messages.
- The basic model of compiler can be represented as in Fig. Q.17.1.

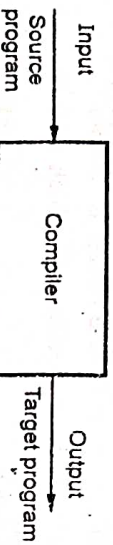


Fig. Q.17.1 Compiler

6.9 : Phases of Compiler**Q.18 Explain various phases of compiler.**

Ans. : 1. **Lexical Analysis** :

- The lexical analysis is also called **scanning**.
- It is the phase of compilation in which the complete source code is scanned and your source program is broken up into group of strings called token.
- A token is a sequence of characters having a collective meaning.
- For example if some assignment statement in your source code is as follows,

total = count + rate * 10

- Then in lexical analysis phase this statement is broken up into series of tokens as follows.

1. The identifier total
2. The assignment symbol
3. The identifier count
4. The plus sign
5. The identifier rate
6. The multiplication sign
7. The constant number 10

- The blank characters which are used in the programming statement are eliminated during the lexical analysis phase.

2. Syntax Analysis :

- The syntax analysis is also called **parsing**.
- In this phase the tokens generated by the lexical analyser are grouped together to form a hierarchical structure.
- The syntax analysis determines the structure of the source string by grouping the tokens together.
- The hierarchical structure generated in this phase is called parse tree or syntax tree.

- For the expression total = count + rate * 10 the parse tree can be generated as shown in Fig. Q.18.1.

- In the statement 'total

= count + rate * 10', first of all rate * 10 will be considered because in arithmetic expression the multiplication operation should be performed before the

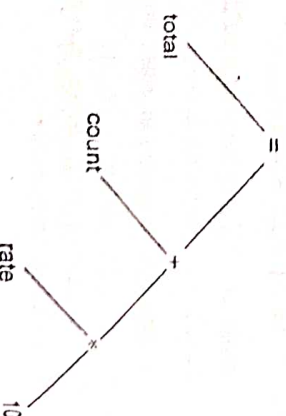


Fig. Q.18.1 Parse tree for total = count + rate * 10

addition. And then the addition operation will be considered. For building such type of syntax tree the production rules are to be designed. The rules are usually expressed by context free grammar.

For the above statement the production rules are -

- (1) $E \rightarrow \text{identifier}$
- (2) $E \rightarrow \text{number}$
- (3) $E \rightarrow E1 + E2$
- (4) $E \rightarrow E1 * E2$
- (5) $E \rightarrow (E)$

where E stands for an expression.

- By rule (1) count and rate are expressions and
- by rule (2) 10 is also an expression.
- By rule (4) we get $\text{rate} * 10$ as expression.
- And finally $\text{count} + \text{rate} * 10$ is an expression.

3. Semantic Analysis

- Once the syntax is checked in the syntax analyser phase the next phase i.e. the semantic analysis determines the meaning of the source string.
- For example meaning of source string means matching of parenthesis in the expression, or matching of if ...else statements or performing arithmetic operations of the expressions that are type compatible, or checking the scope of operation.

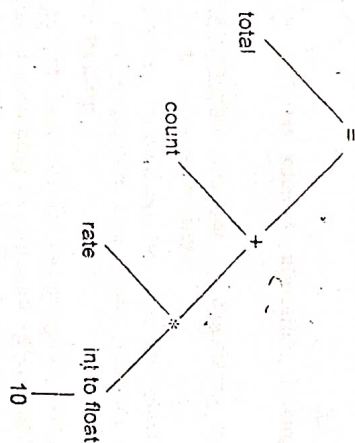


Fig. Q.18.2 Semantic analysis

4. Intermediate Code Generation

- The intermediate code is a kind of code which is easy to generate and this code can be easily converted to target code.
- This code is in variety of forms such as *three address code*, *quadruple*, *triple*, *postfix*. Here we will consider an intermediate code in three address code form. This is like an assembly language.
- The three address code consists of instructions each of which has at the most three operands. For example,

```

t1 := int to float (10)
t2 := rate * t1
t3 := count + t2
total := t3
  
```

- There are certain properties which should be possessed by the three address code and those are,
- 1. Each three address instruction has at the most one operator in addition to the assignment. Thus the compiler has to decide the order of the operations devised by the three address code.
- 2. The compiler must generate a temporary name to hold the value computed by each instruction.
- 3. Some three address instructions may have fewer than three operands for example first and last instruction of above given three address code, i.e.

```

t1 := int_to_float (10)
total := t3
  
```

5. Code Optimization

- The code optimization phase attempts to improve the intermediate code.
- This is necessary to have a **faster executing code or less consumption of memory**.
- Thus by optimizing the code the overall running time of the target program can be improved.

6. Code Generation

- In code generation phase the target code gets generated. The intermediate code instructions are translated into sequence of machine instructions.

```
MOV rate, R1
MUL #10, R1
MOV count, R2
ADD R2, R1
MOV R1, total
```

Explanation of code

Variable 'rate' is moved to register R1.

Then R1 is multiplied with value 10 and result is stored in R1. $\therefore R1 = \text{rate} * 10$.

The value in 'count' is moved to reg. R2 then it is added with R1.

$\therefore R1 = \text{count} + \text{rate} * 10.0$

Finally contents of register R1 is moved to identifier 'total'.

Source program

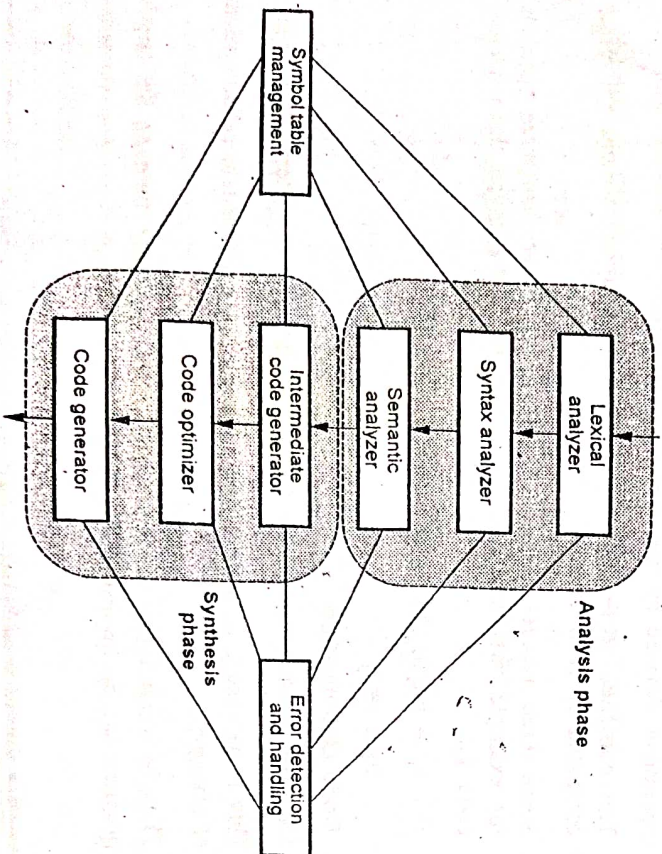


Fig. Q.18.3 Phases of compiler

Q.19 What is compiler? Explain any two phases of compiler with suitable diagram

Ans. : Compiler : Refer Q.17.

Phases of Compiler : Refer Q.18.

Q.20 Consider input "d = a + b * 2;" and show the output of each phase of compiler with suitable diagram.

[SPPU : April-18, In Sem, Marks 5]

Ans. :

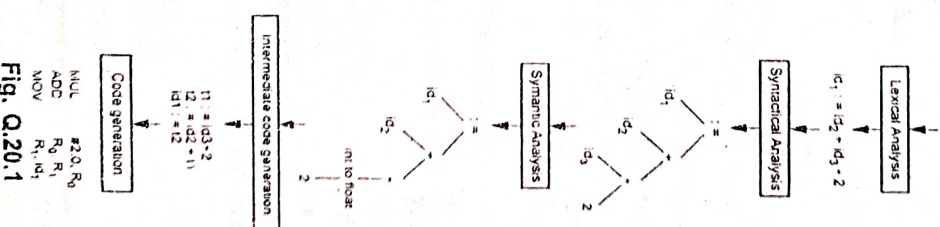


Fig. Q.20.1

6.10 : Loaders

Q.21 Define Loader.

Ans. : Loader is a utility program which takes object code as input prepares it for execution and loads the executable code into the memory. Thus loader is actually responsible for initiating the execution process.

Q.22 Discuss four different functions of loader.

[SPPU : April-18, Marks 4]

OR What are types of loaders? Discuss four different functions of loaders.

[SPPU : May-19, Marks 4]

Ans. : Types of Loaders : Based on various functionalities of the loader, there are various types of loaders, which are known as loading schemes. Various schemes are -

- (1) General Loader Scheme
- (2) Subroutine Linkage
- (3) Relocating Loaders
- (4) Direct Linkage Loader

Functions of Loader :

The loader is responsible for the activities such as allocation, linking, relocation and loading

- 1) It allocates the space for program in the memory, by calculating the size of the program. This activity is called **allocation**.
- 2) It resolves the symbolic references (code/data) between the object modules by assigning all the user subroutine and library subroutine addresses. This activity is called **linking**.
- 3) There are some address dependent locations in the program, such as address constants must be adjusted according to allocated space, such activity done by loader is called **relocation**.
- 4) Finally it places all the machine instructions and data of corresponding programs and subroutines into the memory. Thus program now becomes ready for execution, this activity is called **loading**.

6.11 : General Loader Scheme

Q.23 Explain "General loading scheme (using suitable diagram) with advantages and disadvantages.

[SPPU : Dec-18, March-20, Marks 5, April-19, Marks 6]

Ans. : • In this loader scheme, the source program is converted to object program by some translator (assembler).
• The loader accepts these object modules and puts machine instruction and data in an executable form at their assigned memory.

- The loader occupies some portion of main memory.

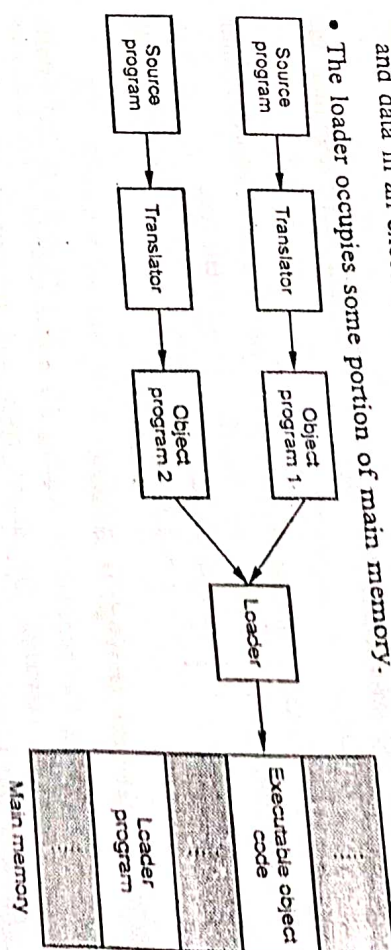


Fig. Q.23.1 General loader scheme

Advantages :

1. The program need not be retranslated each time while running it.
2. It is possible to write source program with multiple programs and multiple languages.

Disadvantages :

1. Some portion of memory is occupied by the loader program, hence memory is wasted.
2. If the program is modified it has to be retranslated.

6.12 : Subroutine Linkages

Q.24 What are subroutine linkages ? What are benefits of using it ?

[SPPU : May-19, Marks 4]

Ans. : • A program consists of main program and set of functions(subroutines). These functions may reside in different files.

• The problem of subroutine linkage is : When a main program wants to call subroutine the subroutine B and if subroutine B resides in some another file then the assembler will not know the address of B and declare it as undefined symbol.

• To realize the interaction of A and B must contain public definition and external references.

• Subroutine linkage allows to establish the reference between main program and various subroutines residing in different files.

6.13 : Linkers

Q.25 Explain the concept of linkers.

Ans. : • During the process of linking the object module is created. This object module contains the target code and information about other programs and library routines that are required to call during the program execution.

• Linker extracts the information present in the object module, locates the required programs and library routines and combine them with the target code and produce a program which a machine can understand. Such a program is called binary program.

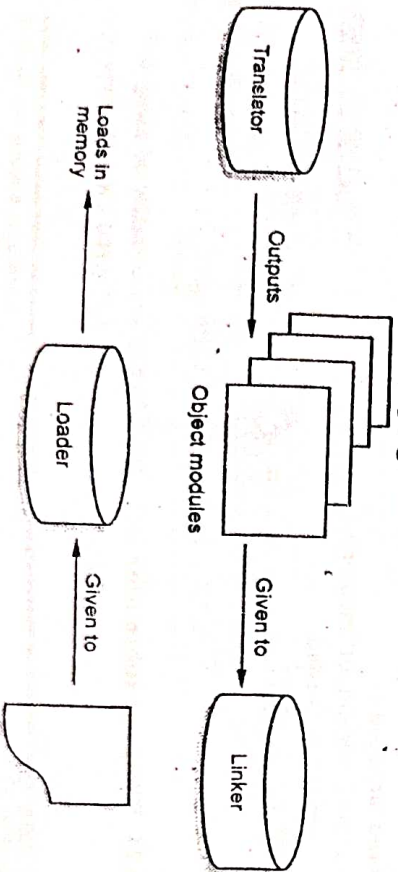


Fig. Q.25.1 Phases of program execution

• The loader then loads this binary program in the memory. (Refer Fig. Q.25.1)

6.14 : Relocation and Linking

Q.26 Explain relocation with suitable example.

Ans. : Program relocation is the process of changing the addresses used in the address sensitive instructions of a program so that the program can execute correctly from designated area of memory.

The relocation can be performed by either linker or loader -

1. If linked origin differs from the translated origin then relocation must be performed by the linker.
2. If load origin differs the linked origin, relocation must be performed by the loader.

The relocation factor can be calculated using following formula -

$$\text{Relocation factor} = \text{Load origin} - \text{Translated origin}$$

The translator puts the address of particular symbol which is used in the program. This is called the translation time address.

Let A is the symbol used in some program then the translated time address is denoted as -

$$\text{Translated time address} = \text{Translated origin} + \text{Offset of A}$$

Similarly the linked time address can be denoted as

$$\text{Linked time address} = \text{Linked origin} + \text{Offset of A}$$

Another way of denoting linked time address is

$$\text{Linked time address} = \text{Translated time address} + \text{Relocation factor}$$

Thus relocation of the program X is the sum of relocation factor for X and translation time address for every instruction.

Example of Relocation

Suppose there is a program A which contains the set of instructions which require relocation. Let us name this set as X.

The relocation of program A can be performed by computing the relocation factor and adding it to translation time address.

Following information is available -

1. Suppose there is a command linker 800 A, B
Then in this linker command 800 denotes the load origin.
2. Let for the set of instructions X (of program A) that needs to be relocated the relocation table is as follows.

Translated Address
400
438

3. The instruction with translated address 400 contains the address 330 in operand field.
4. The instruction with translated address 438 contains the address 302 in operand field translated origin factor.
5. Calculate relocation factor

$$\text{Relocation factor} = \text{Load origin} - \text{Translated origin}$$

$$= 800 - 500$$

Relocation factor = 300

- 1) For instruction with translated address 400 which contains address 330 in operand field, the relocation address will be -

$$\text{Relocation factor} + \text{Address in operand field}$$

$$= 300 + 330 = 630$$

- 2) For instruction with translated address 438 which contains address 302 in operand field, the relocation address will be -

$$\text{Relocation factor} + \text{Address in operand field}$$

$$= 300 + 302 = 602$$

Thus relocatable addresses are now 630 and 602.

Q.27 What is object module ?

Ans.: The object module is nothing but sequence of records. The purpose of object record is to describe specific aspect of the program.

END...✍

SOLVED MODEL QUESTION PAPER (In Sem)

Operating Systems

T.E. (IT) Semester - V [As Per 2019 Pattern]

[Maximum Marks : 30]

Time : 1 Hour]

N.B. : i) Attempt Q.1 or Q.2, Q.3 or Q.4.

- ii) Neat diagrams must be drawn wherever necessary.
- iii) Figures to the right side indicate full marks.
- iv) Assume suitable data, if necessary.

Q.1 a) What is kernel ? List different types of kernel. [4]

(Refer Q.8 of Chapter - 1)

b) What are the benefits of resource abstraction ? [3]

(Refer Q.3 of Chapter - 1)

c) List and explain components of Linux. [8]

(Refer Q.13 of Chapter - 1)

OR.

Q.2 a) Describe in detail the functions of OS as a resource manager. [5]

(Refer Q.2 of Chapter - 1)

b) State and explain different services provided by an operating system. (Refer Q.11 of Chapter - 1) [4]

c) Explain the significance of following shell commands

i) in ii) wc iii) umask iv) cut v) grep

(Refer Q.15 of Chapter - 1)

[6]

Q.3 a) Define process. Differentiate between a process and a program. [3]

(Refer Q.1 of Chapter - 2)

b) What should be scheduling criteria for scheduling algorithm ? (Refer Q.23 of Chapter - 2) [4]

c) For the table given below calculate average waiting time and average turnaround time and draw a Gantt Chart illustrating the process execution using following scheduling algorithms.

i) RR (Time slice - 2 units) ii) SJF (non - preemptive)

Process	Arrival Time	Burst Time
P ₁	0	8
P ₂	1	5
P ₃	3	3
P ₄	4	1
P ₅	6	4

(Refer Q.26 of Chapter - 2)

OR

Q.4 a) Explain thread life cycle. (Refer Q.9 of Chapter - 2) [5]

b) Explain the following functions with reference to 'C'.

i) pthread_create() ii) pthread_join() (Refer Q.17 of Chapter - 2) [4]

c) Explain kernel level thread with their advantages and disadvantages. (Refer Q.15 of Chapter - 2) [6]

SOLVED MODEL QUESTION PAPER (End Sem)

T.E. (IT) Semester - V (As Per 2019 Pattern)

Time : $2\frac{1}{2}$ Hours]

[Maximum Marks : 70]

N.B. : i) Attempt Q.1 or Q.2, Q.3 or Q.4, Q.5 or Q.6, Q.7 or Q.8.

ii) Neat diagrams must be drawn wherever necessary.

iii) Figures to the right side indicate full marks.

iv) Assume suitable data, if necessary.

Q.1 a) Explain the following terms :

i) Mutual exclusion ii) Synchronization iii) Race condition (Refer Q.4 of Chapter - 3) [6]

b) State and explain the deadlock recovery methods. (Refer Q.34 of Chapter - 3) [6]

c) Explain with an appropriate example, how resource allocation graph determines a deadlock. (Refer Q.26 of Chapter - 3) [6]

OR

Q.2 a) Write a pseudo code for producer-consumer problem using semaphore. (Refer Q.17 of Chapter - 3) [6]

b) Consider the following snapshot of the system.

Allocation				Max			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	0	1	0	7	5	3	3	3	2
P2	2	0	0	3	2	2			

P3	3	0	2	9	0	2		
P4	2	1	1	2	2	2		
P5	0	0	2	4	3	3		

Answer the following questions using Banker's algorithm.

a) What are the values of Need Matrix ?

b) Is the system in the safe mode ? If yes, what is the safe sequence ?

c) If a request from process P1 arrives for (1, 0, 2), can the request be granted immediately ? (Refer Q.31 of Chapter - 3) [12]

Q.3 a) What is fragmentation ? Explain types of fragmentation. (Refer Q.7 of Chapter - 4) [5]

b) For the following reference string.

5, 6, 7, 8, 5, 6, 9, 5, 6, 7, 8, 9, 6, 7, 4, 9, 8

Count the number of page fault occur with 3 frames using FIFO, optimal and LRU page replacement methods. Discuss the result. [12]

(Refer Q.27 of Chapter - 4) OR

Q.4 a) What is paging ? Explain in details. (Refer Q.14 of Chapter - 4) [5]

b) For the following reference string. 6, 5, 1, 2, 5, 3, 5, 4, 2, 3, 6, 3, 2, 1, 2 Count the number of page faults that occur with 3 frames using FIFO, Optimal and LRU page replacement methods. Discuss the result. [12]

(Refer Q.28 of Chapter - 4)

Q.5 a) Suppose that a disk drive has 400 cylinders, numbered 0 to 399. The drive is currently serving a request at cylinder 160. The queue of pending requests, in FIFO order, is 370, 30, 390, 130, 310, 170, 340, 180. Starting from the current head position, what is the total distance (in cylinders) that the disk arm moves to satisfy all the pending requests, for each of the following disk scheduling algorithms ?

i) FCFS ii) SSTF iii) SCAN iv) LOOK (Assume that the head is initially moving towards track number 399). [12]

(Refer Q.13 of Chapter - 5)

b) Discuss the following with respect to file system : file operation (Refer Q.15 of Chapter - 5) [6]

OR

Q.6 a) Explain SSTF and LOOK disk scheduling algorithm. (Refer Q.7 of Chapter - 5) [8]

b) Draw and explain file system architecture. (Refer Q.17 of Chapter - 5) [6]

c) Enlist the characteristics of block and character devices? Explain each with suitable example. (Refer Q.1 of Chapter - 5) [4]

Q.7 a) With syntax and example types of assembly language statements. (Refer Q.5 of Chapter - 6) [6]

b) Explain "General loading scheme (using suitable diagram) with advantages and disadvantages. (Refer Q.23 of Chapter - 6) [6]

c) Explain relocation with suitable example. (Refer Q.26 of Chapter - 6) [5]

OR

Q.8 a) Explain in detail Operation code table and symbol table. (Refer Q.9 of Chapter - 6) [6]

b) Explain various phases of compiler. (Refer Q.18 of Chapter - 6) [8]

c) Explain task of analysis phase and synthesis phase. (Refer Q.7 of Chapter - 6) [3]

END...✍